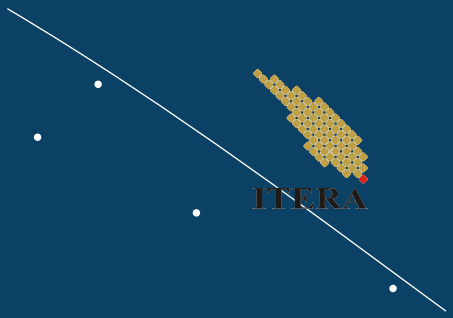
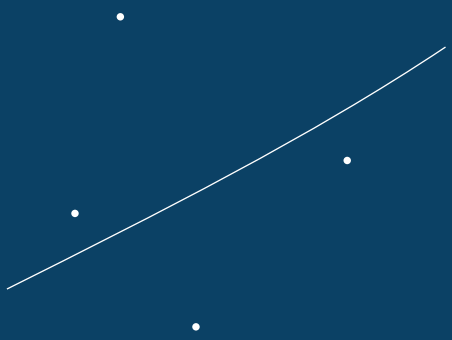


MODUL
PRAKTIKUM

MA25-21008



Program Studi Matematika
Fakultas Sains
Institut Teknologi Sumatera

Pemrograman untuk Ilmu Data dan Komputasi

MA25-21008

Semester Ganjil 2026/2027

Tim Pengajar

Rifky Fauzi
Triyana Muliawati, S.Si., M.Si.

Modul praktikum disusun untuk mendampingi materi kuliah, Lembar Kerja Mandiri, pre-test, dan persiapan UTS.

Contents

Identitas dan Cara Menggunakan Modul **iii**

1 Lingkungan Kerja Python **1**
- 1.1 Lingkungan kerja . . . 1
- 1.2 Struktur folder praktikum . . . 1
- 1.3 Pemeriksaan awal . . . 1

2 Praktikum 1: Python Dasar dan Alur Logika **3**
- 2.1 Tipe data dan operasi . . . 3
- 2.2 Percabangan sebagai representasi kondisi matematika . . . 3
- 2.3 Perulangan . . . 4
- 2.4 None, break, dan continue . . . 4
- 2.5 Loop bersarang . . . 4

3 Praktikum 2: Fungsi, Subprogram, dan Rekursi **6**
- 3.1 Subprogram dan fungsi . . . 6
- 3.2 Empat bentuk fungsi . . . 6
- 3.3 print, return, dan None . . . 6
- 3.4 Multiple return, tuple, dan unpacking . . . 7
- 3.5 Scope lokal dan global . . . 7
- 3.6 Komposisi dan fungsi sebagai objek . . . 7
- 3.7 Lambda . . . 8
- 3.8 Rekursi dasar . . . 8

4 Praktikum 3: Representasi Algoritma dan Struktur Data **10**
- 4.1 Pseudocode . . . 10
- 4.2 List, tuple, dan dictionary . . . 10
- 4.3 List comprehension . . . 10
- 4.4 NumPy array . . . 11
- 4.5 Loop dua dimensi dan matriks sebagai list . . . 11
- 4.6 Slicing dan beda hingga . . . 11

5 Praktikum 4: Debugging, Testing, Terminasi, dan Rekursi **13**
- 5.1 Debugging: kode benar secara sintaks belum tentu benar secara logika . . . 13
- 5.2 Exception handling . . . 13
- 5.3 Testing dengan assert . . . 13
- 5.4 Halt atau terminasi . . . 14
- 5.5 Rekursi dan call stack . . . 14
- 5.6 Iterasi Newton untuk akar kuadrat . . . 14

6 Praktikum 5: Kompleksitas Algoritma dan Sorting Dasar **16**
- 6.1 Dari program ke fungsi banyak kerja . . . 16
  - 6.1.1 Contoh pertama: menjumlahkan $1 + 2 + \cdots + n$ . . . 16
  - 6.1.2 Algoritma lain untuk persoalan yang sama . . . 17
  - 6.1.3 Praktik langsung . . . 18
- 6.2 Struktur loop dan banyak eksekusi . . . 18
  - 6.2.1 Dua loop bersarang . . . 18

6.2.2 Dua loop yang berurutan . . . 18
6.2.3 Loop segitiga . . . 19
6.2.4 Loop yang membagi dua . . . 19
6.3 Mengukur waktu dengan `timeit` . . . 20
6.4 Anagram: satu masalah, dua algoritma . . . 20
6.4.1 Cara 1: urutkan lalu bandingkan . . . 20
6.4.2 Cara 2: hitung frekuensi karakter . . . 21
6.4.3 Uji kedua algoritma . . . 21
6.5 Sorting dasar dengan penghitung operasi . . . 22
6.5.1 Selection sort . . . 22
6.5.2 Insertion sort . . . 23
6.5.3 Bubble sort . . . 23
6.6 Eksperimen pertumbuhan . . . 24

7 Praktikum 6: Searching, Merge Sort, dan Benchmark 26
7.1 Sequential dan binary search . . . 26
7.2 Tracing binary search . . . 26
7.3 Merge sort . . . 26
7.4 Benchmark . . . 27

8 Praktikum 7: Aljabar Linier Komputasional 28
8.1 Vektor: list dan array . . . 28
8.2 Element-wise dan matrix multiplication . . . 28
8.3 Perkalian matriks manual . . . 28
8.4 Transpose dan invers . . . 28
8.5 Sistem persamaan linier dan residual . . . 29
8.6 Nilai eigen . . . 29
8.7 Data sebagai matriks . . . 29

9 Latihan Terpadu Menjelang UTS 31

10 Praktikum 8: Regresi Linear Berbasis Aljabar Linier 33

11 Praktikum 9: Klasifikasi k-Nearest Neighbor 34

12 Praktikum 10: k-Means dan Mini Proyek Data 35

Daftar Pustaka Singkat 36

Identitas dan Cara Menggunakan Modul

Mata kuliah	Pemrograman untuk Ilmu Data dan Komputasi
Kode	MA25-21008
Program studi	Matematika
Fakultas	Fakultas Sains
Institusi	Institut Teknologi Sumatera
Perangkat lunak utama	Python 3, Jupyter Notebook/JupyterLab, Google Colab, atau IDE Python lain
Pustaka utama	Python standard library, NumPy, matplotlib, pandas pada materi setelah UTS
Dokumen pendamping	Materi pertemuan dan Lembar Kerja Mandiri (LKM)

Arah penggunaan

Modul ini bukan kumpulan sintaks. Setiap praktikum menghubungkan tiga hal: persoalan yang hendak diselesaikan, struktur algoritma, dan implementasi Python. Pada bagian sebelum UTS, latihan disusun mengikuti cakupan LKM: prediksi perilaku program, penelusuran alur, penulisan kode, debugging, analisis algoritma, dan komputasi matriks.

Untuk setiap praktikum, mahasiswa disarankan bekerja dengan urutan berikut.

1. Baca tujuan dan konsep inti sebelum membuka editor.
2. Kerjakan prediksi atau perhitungan manual untuk contoh kecil.
3. Tulis algoritma atau pseudocode jika masalah memiliki beberapa tahap.
4. Implementasikan program Python dengan nama variabel yang jelas.
5. Uji program pada beberapa masukan, termasuk kondisi batas.
6. Bandingkan hasil komputasi dengan hasil manual, identitas matematika, atau sifat algoritma yang diketahui.
7. Simpan kode dan hasil secara rapi agar dapat dijalankan ulang.

Peta praktikum

P	**Topik**	**Konsep utama**
1	Python dasar dan alur logika	tipe data, konversi, operasi, input/output, if, for, while, `None`, `break`, `continue`, loop bersarang

2	Fungsi, subprogram, dan rekursi	parameter, argumen, `return`, tuple/unpacking, scope, komposisi, fungsi sebagai objek, lambda, rekursi dasar
3	Representasi algoritma dan struktur data	pseudocode, list, tuple, dictionary, list comprehension, array NumPy, slicing, loop dua dimensi
4	Debugging, testing, dan terminasi	error logika, exception, `assert`, `print`/`return`/`None`, halt, rekursi, iterasi Newton
5	Kompleksitas dan sorting dasar	jumlah operasi, Big-O, pertumbuhan fungsi, selection/insertion/bubble sort, time-space trade-off
6	Searching, merge sort, dan benchmark	sequential/binary search, merge sort, tracing, invariant, `timeit`, benchmark yang adil
7	Aljabar linier komputasional	vektor, matriks, element-wise, matmul, transpose, invers, SPL, residual, norma, nilai eigen, data sebagai matriks
8	Regresi linear	matriks desain, least squares, residual, visualisasi
9	k-Nearest Neighbor	jarak, tetangga terdekat, klasifikasi sederhana, evaluasi
10	k-Means dan proyek data	centroid, iterasi, stopping rule, reproducibility, rancangan mini proyek

Bab 1

Lingkungan Kerja Python

Tujuan praktikum

Mahasiswa mampu menyiapkan lingkungan kerja Python, membuat struktur folder sederhana, menyimpan kode dan hasil, serta menjalankan ulang pekerjaan tanpa bergantung pada keadaan notebook sebelumnya.

1.1 Lingkungan kerja

Praktikum dapat dikerjakan menggunakan Jupyter Notebook, JupyterLab, Google Colab, Spyder, Visual Studio Code, atau IDE Python lain. Pilihan alat tidak mengubah logika program. Yang perlu dijaga adalah versi Python, pustaka yang digunakan, lokasi data, dan urutan eksekusi.

1.2 Struktur folder praktikum

Contoh sederhana:

```
praktikum_01/
  praktikum_01.ipynb
  data/
  output/
  README.md
```

Pada tahap awal, satu notebook sudah cukup. Pemisahan fungsi ke file lain dilakukan ketika materi fungsi dan subprogram telah dibahas.

1.3 Pemeriksaan awal

```
import sys
print(sys.version)
print(2 + 3)
print(type(2 + 3))
```

Latihan persiapan

1. Buat satu folder praktikum dengan struktur di atas.
2. Jalankan Python dan catat versinya.
3. Buat notebook yang menampilkan nama, NIM, dan hasil 2^{10}.
4. Ubah urutan eksekusi dua sel yang saling bergantung. Jelaskan mengapa notebook yang tidak dijalankan dari awal dapat memberi hasil yang membingungkan.
5. Restart kernel, lalu jalankan seluruh sel dari atas. Pastikan hasil tetap sama.
6. Buat `README.md` berisi cara menjalankan notebook dari awal.
7. Simpan satu keluaran teks ke folder `output/`.
8. Tuliskan satu aturan penamaan file yang akan Anda gunakan sepanjang semester.

Bab 2

Praktikum 1: Python Dasar dan Alur Logika

Tujuan praktikum

Mahasiswa mampu membaca dan menulis ekspresi Python, membedakan tipe data dasar, melakukan konversi tipe, menggunakan percabangan dan perulangan, serta menelusuri perubahan nilai variabel selama program berjalan.

2.1 Tipe data dan operasi

Empat tipe yang sering muncul pada awal perkuliahan adalah `int`, `float`, `bool`, dan `str`. Python menentukan perilaku operator berdasarkan tipe operand. Karena itu dua ekspresi yang tampak mirip dapat memiliki arti berbeda.

```
print(1 + True)       # bool berperilaku seperti 0/1 pada aritmetika
print('1' + '0')      # konkatenasi string
print(int('1') + 0)   # konversi eksplisit
print(3 / 2)          # pembagian biasa
print(3 // 2)         # floor division
print(bool('0'))      # string tidak kosong -> True
```

Prinsip membaca ekspresi

Sebelum menjalankan kode, tentukan tipe setiap operand, arti operator untuk tipe tersebut, lalu prediksi tipe dan nilai hasil. Jika operator tidak didefinisikan untuk pasangan tipe itu, prediksi error.

2.2 Percabangan sebagai representasi kondisi matematika

Untuk persamaan kuadrat $ax^2 + bx + c = 0$, diskriminan $D = b^2 - 4ac$ menentukan jenis akar. Struktur `if-elif-else` menerjemahkan kondisi matematika tersebut menjadi alur program.

```
a, b, c = 1.0, -3.0, 2.0
D = b**2 - 4*a*c

if D > 0:
    jenis = "dua akar real"
elif D == 0:
    jenis = "akar real kembar"
else:
```

```
    jenis = "akar kompleks"

print(D, jenis)
```

2.3 Perulangan

`for` cocok ketika himpunan nilai yang akan dilalui sudah jelas. `while` cocok ketika pengulangan bergantung pada kondisi yang berubah selama program berjalan.

```
data = [3, -2, 0, 7, -5, 4, 9]
total_positif = 0

for x in data:
    if x > 0:
        total_positif += x

print(total_positif)
```

2.4 None, break, dan continue

`None` dapat dipakai untuk menyatakan bahwa sebuah nilai tidak tersedia. Ini berbeda dari nol atau nilai ekstrem. `continue` melewati sisa langkah pada iterasi saat ini, sedangkan `break` menghentikan loop.

```
data = [2.5, None, -1.0, 4.2, None, 100.0, 3.1]
valid = []

for x in data:
    if x is None:
        continue
    if abs(x) > 50:
        break
    valid.append(x)

print(valid)
```

2.5 Loop bersarang

Loop bersarang diperlukan ketika proses memiliki dua indeks, misalnya membentuk tabel, pola karakter, atau matriks sederhana.

```
n = 5
for i in range(1, n + 1):
    for j in range(i):
        print('*', end='')
    print()
```

Jumlah karakter yang dicetak adalah $1 + 2 + \cdots + n = n(n+1)/2$.

Latihan Praktikum 1

1. Prediksi nilai dan tipe untuk `1 + True`, `True + True`, `'1' + '0'`, `'1' + 0`, `int('1') + 0`, `2**3.0`, dan `bool('0')`. Verifikasi setelah menulis prediksi.
2. Tulis program yang menerima bilangan bulat positif dan menampilkan bentuk desimal, biner, oktal, dan heksadesimal.
3. Untuk $x = 2.4$ dan $y = -1.2$, hitung $f(x, y) = \dfrac{e^{x-y} + \sqrt{x^2 + y^2}}{1 + |xy|}$ menggunakan modul `math`.
4. Buat program klasifikasi akar persamaan kuadrat berdasarkan diskriminan. Uji tiga kasus berbeda.
5. Implementasikan fungsi potongan secara langsung dengan `if-elif-else`: $x^2 + 1$ untuk $x < 0$, $2x + 1$ untuk $0 \le x < 3$, dan $\sqrt{x + 6}$ untuk $x \ge 3$.
6. Dari list `[3,-2,0,7,-5,4,9]`, hitung jumlah elemen positif, jumlah kuadrat elemen negatif, dan rata-rata elemen nonnol.
7. Gunakan `continue` untuk mengabaikan `None` dan hitung rata-rata data valid.
8. Gunakan `break` untuk menghentikan pemrosesan saat $|x| > 50$. Catat nilai akumulator pada setiap iterasi.
9. Buat segitiga bintang naik dan turun dengan loop bersarang.
10. Buat tabel perkalian 1 sampai n menggunakan dua loop.
11. Tulis program yang mencari faktor positif terkecil selain 1 dari bilangan $n > 1$ menggunakan `while`.
12. Gunakan faktor terkecil untuk menentukan apakah n prima. Uji 101, 221, dan 997.
13. Hitung $p_n = n \sin(\pi/n)$ untuk $n = 3, 4, 10, 100, 10^4, 10^6$ dan amati kecenderungannya.
14. Buat program yang membaca data sampai pengguna memasukkan kata `stop`. Hitung banyak data dan rata-rata.
15. Buat loop yang mencetak semua pasangan (i, j) dengan $1 \le i \le 4$, $1 \le j \le 4$, dan $i + j$ genap.
16. Jelaskan perbedaan makna data bernilai `0`, `None`, dan 100.0 dalam konteks pengukuran.
17. Tulis satu contoh loop `while` yang pasti berhenti dan satu yang tidak berhenti untuk input positif. Jelaskan alasannya.
18. Buat satu soal prediksi output sendiri yang menggunakan `if`, `for`, dan perubahan nilai variabel; tukarkan dengan teman untuk ditelusuri manual.

Bab 3

Praktikum 2: Fungsi, Subprogram, dan Rekursi

Tujuan praktikum

Mahasiswa mampu mendefinisikan fungsi dengan parameter dan nilai balik, memahami scope, menggunakan tuple/unpacking, menyusun fungsi dari fungsi lain, memperlakukan fungsi sebagai objek, serta mengenali struktur dasar rekursi.

3.1 Subprogram dan fungsi

Subprogram adalah bagian program yang diberi nama dan menjalankan tugas tertentu. Pada Python, fungsi yang dibuat dengan `def` merupakan bentuk subprogram yang paling umum. Fungsi membantu memisahkan persoalan besar menjadi unit yang dapat dipanggil, diuji, dan digunakan kembali.

```
def polinom_sederhana(x):
    return 2 - 3*x + x**2

print(polinom_sederhana(4))
```

Dalam definisi di atas, `x` adalah parameter. Pada pemanggilan `polinom_sederhana(4)`, nilai `4` adalah argumen.

3.2 Empat bentuk fungsi

Secara dasar fungsi dapat dibedakan berdasarkan ada tidaknya argumen dan ada tidaknya nilai balik.

Bentuk	Contoh	Makna
Tanpa argumen, tanpa return	`def salam(): print('Halo')`	melakukan aksi
Tanpa argumen, dengan return	`def pi(): return 3.14159`	menghasilkan nilai tetap
Dengan argumen, tanpa return	`def tampil(x): print(x)`	menerima input lalu menampilkan
Dengan argumen, dengan return	`def f(x): return x**2`	menghasilkan nilai untuk dipakai kembali

3.3 print, return, dan None

```
def g(x):
    return x**2 - 1

def h(x):
    print(g(x))

y = h(3)
print(y)
```

`print` hanya menampilkan nilai. Jika fungsi tidak mencapai `return`, Python mengembalikan `None`. Karena itu hasil fungsi seperti `h` tidak langsung dapat dipakai dalam operasi aritmetika.

3.4 Multiple return, tuple, dan unpacking

```
def stat_ringkas(a, b, c, d):
    nilai = [a, b, c, d]
    minimum = min(nilai)
    maksimum = max(nilai)
    rata = sum(nilai) / len(nilai)
    rentang = maksimum - minimum
    return minimum, maksimum, rata, rentang

mn, mx, r, rg = stat_ringkas(4, 7, 1, 9)
```

Python sebenarnya mengembalikan satu tuple. Unpacking menempatkan elemen tuple ke beberapa variabel.

3.5 Scope lokal dan global

```
x = 10

def f(a):
    x = a + 2
    return x

print(f(5))
print(x)
```

Variabel `x` di dalam fungsi bersifat lokal. Untuk program komputasi, fungsi yang menerima input dan mengembalikan output secara eksplisit biasanya lebih mudah diuji daripada fungsi yang mengubah banyak variabel global.

3.6 Komposisi dan fungsi sebagai objek

Jika $(f \circ g)(x) = f(g(x))$, maka pola yang sama dapat ditulis sebagai fungsi yang memanggil fungsi lain.

```
def proses(x, f):
    return f(x)

def kuadrat(x):
    return x**2

```

```
print(proses(5, kuadrat))
```

Nama `kuadrat` tanpa tanda kurung adalah objek fungsi dan dapat dikirim sebagai argumen.

3.7 Lambda

Lambda sesuai untuk fungsi pendek yang hanya terdiri dari satu ekspresi.

```
f = lambda x: 3*x**2 - 2*x + 1
print(f(2))
```

Jika logika mulai panjang, gunakan fungsi bernama dengan `def`.

3.8 Rekursi dasar

Rekursi terjadi ketika fungsi memanggil dirinya sendiri. Dua komponen wajib adalah *base case* dan langkah yang membawa masalah lebih dekat ke base case.

Untuk faktorial,

$$0! = 1, \qquad n! = n(n-1)!, \quad n \geq 1.$$

```
def faktorial(n):
    if n == 0:
        return 1
    return n * faktorial(n - 1)
```

Untuk $n = 4$, urutan pemanggilan adalah $4 \to 3 \to 2 \to 1 \to 0$, lalu nilai dikembalikan ke arah sebaliknya.

Latihan Praktikum 2

1. Buat masing-masing satu contoh untuk empat bentuk fungsi pada tabel di atas.
2. Tulis `polinom(x, koef)` untuk $p(x) = a_0 + a_1x + \cdots + a_nx^n$ tanpa `numpy.polyval`.
3. Buat `stat_ringkas(a,b,c,d)` yang mengembalikan minimum, maksimum, rata-rata, dan rentang. Lakukan unpacking.
4. Ubah fungsi pada nomor sebelumnya agar menerima satu list data.
5. Prediksi keluaran program scope lokal dan global untuk beberapa nilai argumen.
6. Buat versi fungsi yang memakai `global`; jelaskan mengapa hasilnya dapat bergantung pada riwayat pemanggilan.
7. Tulis `jarak(p,q)` dan `energi(d)=1/(1+d^2)`, lalu buat `energi_dua_titik(p,q)` dengan komposisi.
8. Buat `proses(x,f)` lalu uji dengan fungsi kuadrat, kubik, nilai mutlak, dan fungsi linear.
9. Buat list fungsi `ops=[abs, kuadrat, tambah1]` dan terapkan berurutan pada $x = -3$.
10. Definisikan $f(x) = 3x^2 - 2x + 1$ dengan `lambda`, lalu evaluasi untuk $x = -3, -2, \ldots, 3$.
11. Bandingkan kode `map` dan list comprehension untuk evaluasi fungsi yang sama. Pilih yang lebih mudah dibaca dan jelaskan alasan Anda.
12. Diagnosa kode berikut: `def h(x): print(x**2)` kemudian `h(3)+1`. Mengapa gagal? Perbaiki.
13. Buat fungsi `ringkas_vektor(v)` yang mengembalikan $\|v\|_1$, $\|v\|_2$, $\max_i |v_i|$, dan rata-rata komponen.
14. Buat fungsi rekursif faktorial dan tulis jejak pemanggilan untuk $n = 5$.
15. Tulis fungsi iteratif faktorial. Bandingkan struktur kode dengan versi rekursif.
16. Untuk $a_0 = 1$ dan $a_n = 2a_{n-1} + 1$, tulis fungsi rekursif dan hitung a_0 sampai a_5.
17. Buat fungsi rekursif untuk menghitung jumlah $1 + 2 + \cdots + n$.
18. Buat satu fungsi rekursif yang salah karena tidak memiliki base case yang benar. Jelaskan mengapa program tidak berhenti secara wajar.
19. Tulis sebuah fungsi matematika pilihan Anda dengan input dan output yang jelas, tiga kasus uji, dan satu kondisi batas.
20. Ambil satu program panjang dari Praktikum 1 dan pecah menjadi paling sedikit dua fungsi yang mempunyai tanggung jawab berbeda.

Bab 4

Praktikum 3: Representasi Algoritma dan Struktur Data

Tujuan praktikum

Mahasiswa mampu menulis pseudocode, memilih struktur data sederhana, melakukan iterasi pada list/tuple/dictionary, menggunakan comprehension, dan membentuk serta memproses NumPy array satu dan dua dimensi.

4.1 Pseudocode

Pseudocode memisahkan logika algoritma dari sintaks Python. Bentuk yang digunakan di praktikum:

```
ALGORITMA NamaAlgoritma
INPUT:  data dan syarat masukan
OUTPUT: hasil
BEGIN
    inisialisasi
    IF kondisi THEN
        ...
    ENDIF
    FOR / WHILE ...
        ...
    ENDFOR / ENDWHILE
    RETURN hasil
END
```

4.2 List, tuple, dan dictionary

List cocok untuk koleksi berurutan yang dapat diubah. Tuple juga berurutan tetapi lazim dipakai untuk data yang dianggap tetap atau untuk multiple return. Dictionary menyimpan pasangan `key:value`.

```
nilai = {'A': 82, 'B': 71, 'C': 95, 'D': 64}
for nama, skor in nilai.items():
    print(nama, skor)
```

4.3 List comprehension

```
x = [-2, -1, 0, 1, 2]
y = [3*t**2 - 2*t + 1 for t in x]
```

Comprehension ringkas untuk transformasi sederhana. Untuk logika panjang atau bercabang, loop biasa sering lebih jelas.

4.4 NumPy array

Array NumPy bersifat homogen: elemen memiliki tipe data yang konsisten. Operasi aritmetika pada array dilakukan elemen per elemen secara alami.

```
import numpy as np
x = np.array([1, 4., -2, 7])
print(x.dtype)
print(2*x + 1)
```

4.5 Loop dua dimensi dan matriks sebagai list

```
A = []
for i in range(1, 5):
    baris = []
    for j in range(1, 5):
        baris.append(i + j + i*j)
    A.append(baris)
```

Setelah struktur dipahami, list of lists dapat dikonversi menjadi NumPy array untuk operasi numerik selanjutnya.

4.6 Slicing dan beda hingga

Jika y_i diukur pada waktu t_i, kecepatan rata-rata antar titik dapat ditulis

$$v_i = \frac{y_{i+1} - y_i}{t_{i+1} - t_i}.$$

Dengan NumPy, operasi ini dapat dilakukan menggunakan slicing.

```
v = (y[1:] - y[:-1]) / (t[1:] - t[:-1])
```

Latihan Praktikum 3

1. Tulis pseudocode untuk mencari nilai maksimum dan indeksnya dari sebuah list tanpa `max`.
2. Tulis pseudocode algoritma Euclid untuk $\gcd(a, b)$, lalu trace untuk $(252, 105)$.
3. Buat contoh iterasi list, tuple, dan dictionary. Jelaskan nilai yang dihasilkan pada setiap iterasi.
4. Dari dictionary nilai mahasiswa, hitung rata-rata dan key dengan nilai tertinggi.
5. Dengan list comprehension, hitung $y_i = 3x_i^2 - 2x_i + 1$ untuk $x = [-2, -1, 0, 1, 2]$.
6. Dari matriks list 3×3, ambil kolom terakhir dan diagonal utama tanpa NumPy.
7. Bentuk list berisi dua kali kuadrat elemen kolom tengah dengan comprehension.
8. Konversi list `[1,4.,-2,7]` menjadi NumPy array dan jelaskan `dtype` yang diperoleh.
9. Buat array integer dan float masing-masing panjang 5. Bandingkan hasil pembagian dengan 2.
10. Hitung $y_i = \sin(x_i) + x_i^2$ pada 1000 titik dengan loop Python dan NumPy. Pastikan hasil sama sampai toleransi tertentu.
11. Bentuk $A \in \mathbb{R}^{4\times4}$ dengan $a_{ij} = i + j + ij$ menggunakan dua loop.
12. Tentukan diagonal utama dan jumlah diagonal dari matriks nomor 11 tanpa NumPy.
13. Periksa secara manual dan dengan NumPy apakah matriks nomor 11 simetris.
14. Diberikan data yang mengandung `None`. Abaikan data hilang dan hentikan proses jika $|x| > 50$.
15. Bentuk dictionary skor-z dari dictionary nilai. Gunakan simpangan baku populasi dan verifikasi rata-rata skor-z mendekati nol.
16. Buat array waktu dan posisi sederhana, lalu hitung kecepatan rata-rata menggunakan slicing.
17. Jelaskan mengapa panjang array kecepatan satu lebih kecil dari panjang array posisi.
18. Buat matriks 5×5 dengan $a_{ij} = |i - j|$ menggunakan nested comprehension.
19. Bandingkan hasil loop biasa dan comprehension untuk transformasi data yang sama. Fokus pada keterbacaan, bukan hanya jumlah baris.
20. Tulis pseudocode dan kode Python untuk menghitung frekuensi kemunculan setiap kategori menggunakan dictionary.

Bab 5

Praktikum 4: Debugging, Testing, Terminasi, dan Rekursi

Tujuan praktikum

Mahasiswa mampu menemukan kesalahan logika, membedakan error tipe/scope/kondisi batas, menggunakan exception dan assert secara proporsional, menilai apakah loop atau rekursi berhenti, serta menelusuri algoritma iteratif dan rekursif.

5.1 Debugging: kode benar secara sintaks belum tentu benar secara logika

```
def rata(data):
    total = 0
    for x in data:
        total += x
        return total / len(data)
```

Kode ini valid secara sintaks, tetapi `return` berada terlalu awal. Uji dengan data kecil untuk menunjukkan bug sebelum memperbaikinya.

5.2 Exception handling

```
def rasio(a, b):
    try:
        x = float(a)
        y = float(b)
        return x / y
    except (TypeError, ValueError):
        return None
    except ZeroDivisionError:
        return None
```

Exception handling dipakai ketika kegagalan memang merupakan kondisi yang mungkin muncul dari input atau operasi. Jangan menangkap semua exception tanpa alasan karena bug pemrograman dapat tersembunyi.

5.3 Testing dengan assert

```python
def jarak_1d(x, y):
    return abs(x - y)

assert jarak_1d(3, 3) == 0
assert jarak_1d(3, 8) == 5
assert jarak_1d(-2, 3) == 5
```

Kasus uji yang baik mencakup kasus normal, kondisi batas, dan kasus yang hasilnya sudah diketahui.

5.4 Halt atau terminasi

Algoritma dikatakan berhenti untuk sebuah input jika eksekusinya selesai setelah sejumlah langkah berhingga. Untuk loop, salah satu cara memberi alasan terminasi adalah menemukan ukuran bilangan bulat nonnegatif yang berkurang menuju nol.

```python
i = n
while i > 0:
    i = i // 2
```

Untuk $n > 0$, nilai `i` mengecil hingga nol.

5.5 Rekursi dan call stack

Pada rekursi, setiap pemanggilan menunggu hasil pemanggilan yang lebih kecil. Base case menghentikan rantai pemanggilan.

5.6 Iterasi Newton untuk akar kuadrat

Untuk $f(x) = x^2 - n$, Newton-Raphson memberikan

$$x_{k+1} = x_k - \frac{x_k^2 - n}{2x_k} = \frac{1}{2}\left(x_k + \frac{n}{x_k}\right).$$

Contoh implementasi dengan jumlah iterasi tetap:

```python
def square_root(n):
    root = n / 2
    for k in range(20):
        root = 0.5 * (root + n / root)
    return root
```

Kriteria berhenti dapat diperbaiki dengan toleransi $|x_{k+1} - x_k| < \varepsilon$.

Latihan Praktikum 4

1. Temukan dan perbaiki bug pada fungsi rata-rata dengan `return` di dalam loop.
2. Buat dua test case yang pasti gagal pada versi salah dan lulus pada versi benar.
3. Diagnosa fungsi yang mencoba mengubah variabel global tanpa deklarasi `global`. Berikan perbaikan dengan dan tanpa variabel global.
4. Tambahkan pemeriksaan pada fungsi invers 2×2 agar matriks singular ditolak.
5. Tulis lima `assert` untuk fungsi jarak Euclid dua vektor.
6. Uji sifat simetri $d(x, y) = d(y, x)$ dan kasus $d((0, 0), (3, 4)) = 5$.
7. Tulis `rasio(a,b)` yang menangani `ValueError`, `TypeError`, dan `ZeroDivisionError`.
8. Jelaskan perbedaan `print(g(x))`, `return g(x)`, dan fungsi tanpa `return`.
9. Tentukan apakah loop `i=i//2`, `i=i-2`, dan `i=i+1` berhenti untuk semua input positif.
10. Untuk loop `i=i-2`, klasifikasikan input yang mencapai kondisi tertentu dan yang tidak.
11. Definisikan ukuran yang menurun untuk membuktikan terminasi loop pembagian dua.
12. Tulis fungsi rekursif untuk $a_0 = 2$, $a_n = 3a_{n-1} - 2$. Hitung a_1 sampai a_6.
13. Tulis fungsi rekursif untuk $u_0 = 1$, $u_1 = 3$, $u_n = u_{n-1} + 2u_{n-2}$. Gambar pohon pemanggilan untuk u_4.
14. Bandingkan faktorial rekursif dan iteratif untuk $n = 5$. Tulis call stack versi rekursif.
15. Jelaskan mengapa rekursi Fibonacci naif menghitung submasalah yang sama berkali-kali.
16. Gunakan iterasi Newton untuk menghitung $\sqrt{2}$, $\sqrt{9}$, dan $\sqrt{4563}$.
17. Modifikasi Newton agar berhenti berdasarkan toleransi, lalu kembalikan juga banyak iterasi.
18. Uji Newton untuk nilai awal berbeda. Apakah semua pilihan nilai awal layak?
19. Buat satu contoh fungsi yang gagal pada kondisi batas meskipun bekerja pada input biasa. Tulis test yang menangkap kegagalan tersebut.
20. Pilih satu fungsi dari praktikum sebelumnya dan tulis sekurangnya enam test case yang mencakup normal, batas, dan input tidak valid.

Bab 6

Praktikum 5: Kompleksitas Algoritma dan Sorting Dasar

Tujuan praktikum

Mahasiswa mampu membaca program sebagai rangkaian kerja komputasi, menghitung banyak eksekusi operasi utama sebagai fungsi ukuran input, menghubungkannya dengan Big-O, membedakan jumlah operasi dari waktu eksekusi aktual, serta menguji beberapa algoritma pada Python.

6.1 Dari program ke fungsi banyak kerja

Pada praktikum ini kita tidak hanya melihat keluaran program. Kita juga bertanya:

> Jika ukuran input menjadi lebih besar, seberapa cepat banyak pekerjaan program bertambah?

Kita tulis $W(n)$ untuk **fungsi banyak kerja** terhadap ukuran input n. Dalam satu analisis, operasi yang dihitung harus dinyatakan dengan jelas. Misalnya:

- banyak eksekusi baris terdalam;
- banyak perbandingan;
- banyak pertukaran;
- atau banyak operasi aritmetika yang tampak pada kode.

Catatan

$W(n)$ bukan waktu dalam detik. Nilai $W(n)$ diperoleh dari struktur algoritma. Waktu aktual baru diukur kemudian dengan `timeit`. Keduanya berhubungan, tetapi bukan hal yang sama.

6.1.1 Contoh pertama: menjumlahkan $1 + 2 + \cdots + n$

Mulai dari versi yang langsung mengikuti definisi penjumlahan.

```
def jumlah_loop(n):
    total = 0
    for i in range(1, n + 1):
        total = total + i
    return total
```

Jika yang dihitung adalah operasi yang tampak pada kode:

- `total = 0`: 1 assignment;
- `n + 1`: 1 penjumlahan, dihitung sekali saat membentuk `range`;
- `total = total + i`: 1 penjumlahan dan 1 assignment, dilakukan n kali;
- `return total`: 1 return.

Dengan model hitung tersebut,

$$W_{\text{loop}}(n) = 1 + 1 + 2n + 1 = 2n + 3,$$

maka

$$W_{\text{loop}}(n) = O(n).$$

Sekarang tambahkan beberapa baris yang hanya bersifat tampilan.

```
def jumlah_loop_dekoratif(n):
    print("Halo")
    print("Menghitung jumlah 1 sampai", n)
    print("----------------------------")

    total = 0
    for i in range(1, n + 1):
        total = total + i

    return total
```

Tiga `print` menambah tiga pekerjaan tetap:

$$W_{\text{dekoratif}}(n) = 2n + 6.$$

Pertumbuhannya tetap

$$O(n).$$

Artinya, menambah beberapa baris tetap dapat mengubah jumlah pekerjaan nyata, tetapi tidak mengubah orde pertumbuhan terhadap n.

6.1.2 Algoritma lain untuk persoalan yang sama

Identitas

$$1 + 2 + \cdots + n = \frac{n(n+1)}{2}$$

memberikan algoritma berbeda.

```
def jumlah_rumus(n):
    return n * (n + 1) // 2
```

Pada ekspresi tersebut terdapat penjumlahan, perkalian, pembagian bilangan bulat, dan return. Banyaknya tidak berubah ketika n membesar, sehingga

$$W_{\text{rumus}}(n) = 4 = O(1)$$

dengan konvensi hitung di atas.

Yang penting

Dua program dapat menghasilkan nilai matematika yang sama, tetapi melakukan jumlah pekerjaan yang berbeda:

$$\text{loop} : O(n), \qquad \text{rumus tertutup} : O(1).$$

6.1.3 Praktik langsung

Jalankan kode berikut untuk beberapa nilai n.

```
for n in [5, 10, 100, 1000]:
    a = jumlah_loop(n)
    b = jumlah_rumus(n)
    print(n, a, b, a == b)
```

Sebelum menjalankan program, hitung terlebih dahulu $W_{\text{loop}}(n)$ dan $W_{\text{dekoratif}}(n)$ untuk nilai n yang sama.

6.2 Struktur loop dan banyak eksekusi

Untuk melihat struktur algoritma secara langsung, kita dapat menggunakan counter. Counter ini adalah alat observasi: ia membantu memeriksa berapa kali bagian tertentu dijalankan.

6.2.1 Dua loop bersarang

```
def hitung_persegi(n):
    operasi = 0
    for i in range(n):
        for j in range(n):
            operasi += 1
    return operasi
```

Baris `operasi += 1` dijalankan

$$n \times n = n^2$$

kali. Jadi, jika baris terdalam dipilih sebagai operasi utama,

$$W(n) = n^2, \qquad O(n^2).$$

6.2.2 Dua loop yang berurutan

```
def hitung_berurutan(n):
    operasi = 0

    for i in range(n):
        operasi += 1

    for j in range(n):
        operasi += 1

    return operasi
```

Loop pertama menjalankan baris utama n kali dan loop kedua juga n kali:

$$W(n) = n + n = 2n, \qquad O(n).$$

Dua loop tidak otomatis menghasilkan $O(n^2)$. Yang menentukan adalah apakah loop **bersarang** atau **berurutan**.

6.2.3 Loop segitiga

```python
def hitung_segitiga(n):
    operasi = 0

    for i in range(n):
        for j in range(i + 1):
            operasi += 1

    return operasi
```

Untuk $i = 0$ ada 1 iterasi, untuk $i = 1$ ada 2 iterasi, dan seterusnya:

$$W(n) = 1 + 2 + \cdots + n = \frac{n(n+1)}{2}.$$

Karena suku dominannya adalah $\frac{1}{2}n^2$,

$$W(n) = O(n^2).$$

Verifikasi dengan:

```python
for n in range(1, 8):
    hasil = hitung_segitiga(n)
    rumus = n * (n + 1) // 2
    print(n, hasil, rumus)
```

6.2.4 Loop yang membagi dua

```python
def hitung_bagi_dua(n):
    i = n
    operasi = 0

    while i > 0:
        operasi += 1
        i = i // 2

    return operasi
```

Coba untuk

$$n = 1, 2, 4, 8, 16, 32, 64.$$

Setiap iterasi mengurangi ukuran masalah kira-kira menjadi setengah. Untuk $n \geq 1$, banyak iterasinya

$$\lfloor \log_2 n \rfloor + 1,$$

sehingga

$$W(n) = O(\log n).$$

6.3 Mengukur waktu dengan `timeit`

Jumlah operasi berasal dari analisis algoritma. Kita juga dapat mengukur waktu implementasi, tetapi hasilnya dipengaruhi Python, implementasi fungsi, komputer, proses lain pada sistem, cache, dan bentuk input.

Gunakan beberapa pengulangan dan ambil nilai tengah.

```
from timeit import repeat
from statistics import median

def waktu_per_panggilan(f, n, number=1000):
    hasil = repeat(
        lambda: f(n),
        repeat=7,
        number=number
    )
    return median(hasil) / number
```

Contoh:

```
for n in [10, 100, 1000, 10000]:
    t_loop = waktu_per_panggilan(jumlah_loop, n)
    t_rumus = waktu_per_panggilan(jumlah_rumus, n)

    print(n, t_loop, t_rumus)
```

Catatan

Jangan memasukkan `print` ke dalam fungsi yang sedang dibandingkan waktunya. Input/output ke layar dapat jauh lebih mahal daripada operasi aritmetika dan menutupi perilaku algoritma yang sebenarnya ingin diamati.
timeit tidak menghitung kompleksitas. Ia hanya mengukur waktu implementasi tertentu untuk input tertentu. Big-O diperoleh dari pertumbuhan $W(n)$ ketika n membesar.

6.4 Anagram: satu masalah, dua algoritma

Dua string disebut anagram jika memiliki karakter yang sama dengan frekuensi yang sama, hanya urutannya berbeda.

Contoh:

`listen` dan `silent`.

6.4.1 Cara 1: urutkan lalu bandingkan

```
def anagram_sort(a, b):
    if len(a) != len(b):
        return False

    return sorted(a) == sorted(b)
```

Untuk string panjang n, sorting membutuhkan pertumbuhan sekitar

$$O(n \log n).$$

Perbandingan dua hasil sorting membutuhkan paling banyak $O(n)$, sehingga orde keseluruhan tetap

$$O(n \log n).$$

6.4.2 Cara 2: hitung frekuensi karakter

Untuk latihan ini anggap string hanya berisi huruf kecil `a` sampai `z`.

```
def anagram_frekuensi(a, b):
    if len(a) != len(b):
        return False

    freq = [0] * 26

    for ch in a:
        indeks = ord(ch) - ord('a')
        freq[indeks] += 1

    for ch in b:
        indeks = ord(ch) - ord('a')
        freq[indeks] -= 1

    return all(x == 0 for x in freq)
```

Dua loop masing-masing melalui n karakter. Pemeriksaan terakhir hanya melalui 26 posisi:

$$W(n) \sim n + n + 26,$$

maka

$$O(n).$$

Array frekuensi memakai 26 tempat tambahan. Untuk alfabet yang ukurannya dianggap tetap, kebutuhan ruang tambahan tersebut adalah $O(1)$ terhadap n.

Time-space trade-off

Versi frekuensi menggunakan struktur tambahan untuk menghindari sorting. Ini adalah contoh bahwa algoritma yang lebih cepat dapat memakai representasi data tambahan.

6.4.3 Uji kedua algoritma

```
pasangan = [
    ("listen", "silent"),
    ("matematika", "matemaatik"),
    ("algoritma", "logaritma"),
    ("data", "adat"),
    ("python", "typhon"),
]

for a, b in pasangan:
    print(a, b,
          anagram_sort(a, b),
          anagram_frekuensi(a, b))
```

Buat pula input lebih panjang dan ukur kedua implementasi.

```
def buat_anagram(n):
```

```
    pola = "matematika"
    a = (pola * (n // len(pola) + 1))[:n]
    b = a[::-1]
    return a, b

for n in [100, 1000, 10000]:
    a, b = buat_anagram(n)

    t_sort = median(repeat(
        lambda: anagram_sort(a, b),
        repeat=7, number=100
    )) / 100

    t_freq = median(repeat(
        lambda: anagram_frekuensi(a, b),
        repeat=7, number=100
    )) / 100

    print(n, t_sort, t_freq)
```

Jangan menyimpulkan orde hanya dari satu tabel waktu. Gunakan analisis algoritma untuk menentukan Big-O, lalu gunakan data waktu untuk melihat apakah kecenderungan empiris masuk akal.

6.5 Sorting dasar dengan penghitung operasi

Selain hasil sorting, kita dapat mencatat apa yang dilakukan algoritma.

6.5.1 Selection sort

```
def selection_sort_hitung(data):
    a = data.copy()
    n = len(a)
    perbandingan = 0
    pertukaran = 0

    for i in range(n - 1):
        min_index = i

        for j in range(i + 1, n):
            perbandingan += 1

            if a[j] < a[min_index]:
                min_index = j

        if min_index != i:
            a[i], a[min_index] = a[min_index], a[i]
            pertukaran += 1

    return a, perbandingan, pertukaran
```

Jumlah perbandingan selection sort:

$$(n-1) + (n-2) + \cdots + 1 = \frac{n(n-1)}{2},$$

sehingga

$$O(n^2).$$

Perhatikan bahwa jumlah **pertukaran** tidak harus sama dengan jumlah **perbandingan**.

6.5.2 Insertion sort

```
def insertion_sort_hitung(data):
    a = data.copy()
    perbandingan = 0

    for i in range(1, len(a)):
        key = a[i]
        j = i - 1

        while j >= 0:
            perbandingan += 1

            if a[j] <= key:
                break

            a[j + 1] = a[j]
            j -= 1

        a[j + 1] = key

    return a, perbandingan
```

Jumlah perbandingan insertion sort sangat dipengaruhi susunan awal data. Data yang sudah hampir terurut dapat membutuhkan jauh lebih sedikit kerja daripada data terbalik.

6.5.3 Bubble sort

```
def bubble_sort_hitung(data):
    a = data.copy()
    n = len(a)
    perbandingan = 0
    pertukaran = 0

    for akhir in range(n - 1, 0, -1):
        for i in range(akhir):
            perbandingan += 1

            if a[i] > a[i + 1]:
                a[i], a[i + 1] = a[i + 1], a[i]
                pertukaran += 1

    return a, perbandingan, pertukaran
```

Uji ketiganya pada:

```
data_acak = [8, 3, 7, 4, 9, 2, 6, 5]
data_urut = sorted(data_acak)
data_terbalik = sorted(data_acak, reverse=True)

print(selection_sort_hitung(data_acak))
print(insertion_sort_hitung(data_acak))
```

```
7 print(bubble_sort_hitung(data_acak))
```

Bandingkan hasil, banyak perbandingan, dan banyak pertukaran. Jangan hanya membandingkan waktu.

6.6 Eksperimen pertumbuhan

Gunakan beberapa ukuran input, misalnya

$$n = 100,\ 200,\ 400,\ 800.$$

Untuk setiap ukuran:

1. buat input sebelum pengukuran waktu;
2. gunakan input yang sama ketika dua algoritma dibandingkan;
3. catat jumlah operasi yang relevan;
4. ukur waktu beberapa kali;
5. ambil median;
6. lihat apa yang terjadi ketika n digandakan.

Jika algoritma mendekati $O(n)$, menggandakan n cenderung membuat banyak kerja sekitar dua kali. Untuk $O(n^2)$, banyak kerja cenderung sekitar empat kali. Ini adalah pola pertumbuhan, bukan aturan bahwa waktu di komputer harus tepat 2 atau 4 kali.

Latihan Praktikum 5

1. Buat tabel nilai 1, $\log_2 n$, n, $n \log_2 n$, n^2, n^3, 2^n, dan $n!$ untuk beberapa n kecil. Dari tabel tersebut, susun urutan pertumbuhan dari paling lambat.
2. Jalankan `hitung_persegi(n)` untuk $n = 2, 4, 8, 16$. Bandingkan keluaran dengan n^2.
3. Jalankan `hitung_berurutan(n)` pada nilai n yang sama. Verifikasi bahwa banyak eksekusi baris utama adalah $2n$, bukan n^2.
4. Jalankan `hitung_bagi_dua(n)` untuk $n = 1, 2, 4, \ldots, 1024$. Buat tabel n dan banyak iterasi.
5. Jalankan `hitung_segitiga(n)` untuk $n = 1, \ldots, 10$ dan verifikasi terhadap $n(n+1)/2$.
6. Untuk `jumlah_loop`, hitung $W(n)$ dari kode, lalu cek keluaran untuk $n = 5, 10, 100$. Tunjukkan mengapa pertumbuhannya $O(n)$.
7. Tambahkan tiga `print` seperti pada `jumlah_loop_dekoratif`. Hitung kembali $W(n)$. Jelaskan apa yang berubah dan apa yang tidak berubah.
8. Bandingkan `jumlah_loop` dan `jumlah_rumus` dengan `timeit` untuk minimal lima ukuran n. Pastikan kedua fungsi menghasilkan nilai yang sama sebelum mengukur waktu.
9. Pilih dua operasi list, misalnya indexing dan `pop(0)`. Gunakan `timeit` pada beberapa ukuran list dan catat kecenderungannya.
10. Implementasikan `anagram_sort`. Uji sedikitnya lima pasangan string: anagram, bukan anagram, panjang berbeda, karakter berulang, dan string kosong.
11. Implementasikan `anagram_frekuensi`. Pastikan hasilnya sama dengan `anagram_sort` untuk seluruh test case.
12. Untuk kedua algoritma anagram, tuliskan operasi yang paling sering dilakukan, fungsi banyak kerjanya terhadap n, Big-O waktu, dan kebutuhan ruang tambahan.
13. Gunakan `timeit` untuk membandingkan kedua algoritma anagram pada $n = 10^2, 10^3, 10^4$. Jelaskan mengapa hasil waktu tidak boleh dipakai sebagai definisi kompleksitas.
14. Untuk
$$W_1(n) = 7n^2 + 3n + 40, \qquad W_2(n) = 0.001n^3 + 100n,$$
tentukan orde keduanya dan cari secara numerik rentang n ketika $W_1(n) < W_2(n)$.
15. Trace tiga pass pertama selection sort untuk list `[9,4,7,1,6,3,8,2]`. Setelah setiap pass, tulis bagian yang sudah berada pada posisi final.
16. Jalankan `selection_sort_hitung` pada data terurut, acak, dan terbalik dengan panjang yang sama. Bandingkan banyak perbandingan dan pertukaran.
17. Jalankan `insertion_sort_hitung` pada tiga bentuk data yang sama. Bandingkan banyak perbandingan dan jelaskan perbedaannya.
18. Jalankan `bubble_sort_hitung` pada tiga bentuk data yang sama. Catat banyak perbandingan dan pertukaran.
19. Untuk sekurangnya empat ukuran input, buat tabel berisi n, bentuk input, jumlah operasi, dan waktu aktual untuk selection, insertion, dan bubble sort. Verifikasi seluruh hasil dengan `sorted`.
20. Tulis kesimpulan praktikum yang membedakan dengan jelas: fungsi banyak kerja $W(n)$, Big-O, jumlah operasi yang benar-benar tercatat, dan waktu eksekusi aktual dari `timeit`.

Bab 7

Praktikum 6: Searching, Merge Sort, dan Benchmark

Tujuan praktikum

Mahasiswa mampu menelusuri sequential dan binary search, memahami kebutuhan data terurut pada binary search, mengikuti rekursi merge sort, menggunakan invariant sederhana, dan merancang benchmark yang adil.

7.1 Sequential dan binary search

Sequential search memeriksa elemen satu per satu. Binary search memanfaatkan keterurutan data dan mengurangi interval pencarian sekitar setengah pada setiap langkah.

```
def binary_search(a, target):
    kiri, kanan = 0, len(a) - 1
    while kiri <= kanan:
        tengah = (kiri + kanan) // 2
        if a[tengah] == target:
            return tengah
        if a[tengah] < target:
            kiri = tengah + 1
        else:
            kanan = tengah - 1
    return -1
```

7.2 Tracing binary search

Untuk list terurut `[3,5,6,8,11,12,14,15,17,18]`, catat pada setiap langkah: indeks kiri, kanan, tengah, nilai tengah, dan keputusan berikutnya. Tracing menjelaskan mengapa kompleksitas binary search adalah $O(\log n)$.

7.3 Merge sort

Merge sort membagi list menjadi dua sampai sublist kecil, lalu menggabungkan kembali dua list terurut. Struktur pembagian menghasilkan kedalaman sekitar $\log_2 n$, sedangkan setiap level memproses total sekitar n elemen, sehingga kompleksitasnya $O(n \log n)$.

7.4 Benchmark

Benchmark mengukur implementasi, bukan menggantikan analisis kompleksitas. Gunakan input yang sama, beberapa ukuran data, pengulangan pengukuran, dan hindari pekerjaan tambahan yang hanya dilakukan oleh salah satu algoritma.

Latihan Praktikum 6

1. Trace binary search untuk mencari 8 pada `[3,5,6,8,11,12,14,15,17,18]`.
2. Trace binary search untuk mencari 16 pada list yang sama sampai algoritma menyatakan tidak ditemukan.
3. Implementasikan sequential search dan kembalikan indeks serta banyak perbandingan.
4. Modifikasi binary search agar juga mengembalikan banyak perbandingan.
5. Bandingkan jumlah perbandingan untuk target di awal, tengah, akhir, dan tidak ditemukan.
6. Uji binary search pada data yang belum terurut. Jelaskan mengapa hasilnya tidak dapat dipercaya.
7. Tulis binary search rekursif dan bandingkan alurnya dengan versi iteratif.
8. Gambar tiga level awal pohon rekursi merge sort untuk `[21,1,26,45,29,28,2,9,16,49,39,27,43,34,46,40]`.
9. Tentukan pasangan sublist pertama yang digabung kembali pada merge sort tersebut.
10. Implementasikan fungsi `merge(left,right)` dan uji dengan dua list kecil yang sudah terurut.
11. Implementasikan merge sort rekursif.
12. Bandingkan worst-case bubble, selection, insertion, merge, dan quicksort standar.
13. Tulis invariant sederhana untuk selection sort dan gunakan untuk menjelaskan kebenaran algoritma.
14. Rancang benchmark insertion sort vs merge sort pada ukuran input yang meningkat.
15. Gunakan data acak yang sama untuk kedua algoritma pada setiap ulangan.
16. Bandingkan data acak, terurut, dan hampir terurut.
17. Ambil x_i pada $[0, 2\pi]$ dan hitung $\sin(x_i)+x_i^2$ dengan loop dan NumPy vectorization. Pastikan hasil sama sebelum mengukur waktu.
18. Ulangi benchmark vectorization untuk $n = 10^3, 10^4, 10^5$.
19. Jelaskan mengapa satu pengukuran waktu tidak cukup untuk mengambil kesimpulan.
20. Buat laporan ringkas berisi teori kompleksitas, data waktu, dan interpretasi apakah hasil empiris konsisten dengan teori.

Bab 8

Praktikum 7: Aljabar Linier Komputasional

Tujuan praktikum

Mahasiswa mampu merepresentasikan vektor dan matriks, membedakan operasi element-wise dan perkalian matriks, menghitung transpose dan invers, menyelesaikan SPL, memeriksa residual, menghitung norma dan nilai eigen, serta melihat data sebagai matriks.

8.1 Vektor: list dan array

List Python adalah struktur umum. NumPy array dirancang untuk komputasi numerik dan mendukung operasi vektor secara langsung.

```
import numpy as np
x = np.array([1., -2., 3.])
y = np.array([4., 0., -1.])
print(x + y)
print(np.dot(x, y))
print(np.linalg.norm(x))
```

8.2 Element-wise dan matrix multiplication

Untuk array NumPy, `A*B` berarti perkalian elemen yang bersesuaian, sedangkan `A@B` berarti perkalian matriks.

8.3 Perkalian matriks manual

Untuk memahami definisi $C = AB$ dengan $c_{ij} = \sum_k a_{ik} b_{kj}$, implementasikan perkalian matriks dengan tiga loop sebelum memakai `@`.

8.4 Transpose dan invers

Untuk $A \in \mathbb{R}^{m \times n}$, transpose memiliki ukuran $n \times m$. Untuk

$$A = \begin{pmatrix} a & b \\ c & d \end{pmatrix}, \qquad A^{-1} = \frac{1}{ad - bc} \begin{pmatrix} d & -b \\ -c & a \end{pmatrix},$$

selama $ad - bc \neq 0$.

8.5 Sistem persamaan linier dan residual

Untuk $Ax = b$, solusi numerik $\hat{x}$ perlu diperiksa melalui residual

$$r = b - A\hat{x}, \qquad \|r\|_2.$$

```
A = np.array([[2., 1.], [5., 3.]])
b = np.array([1., 4.])
x = np.linalg.solve(A, b)
r = b - A @ x
print(x, np.linalg.norm(r))
```

8.6 Nilai eigen

Pasangan (λ, v) memenuhi $Av = \lambda v$. Setelah memperoleh hasil dari `np.linalg.eig`, verifikasi numerik dengan norma $\|Av - \lambda v\|_2$.

8.7 Data sebagai matriks

Jika baris mewakili observasi dan kolom mewakili fitur, data dapat ditulis sebagai matriks X. Centering dilakukan dengan mengurangi rata-rata setiap kolom. Matriks $X_c^T X_c$ merangkum hasil kali silang antarfitur dan selalu simetris.

Latihan Praktikum 7

1. Untuk $x = (1, -2, 3)$ dan $y = (4, 0, -1)$, hitung manual $x + y$, $2x - y$, $x \cdot y$, $\|x\|_2$, dan $\|x - y\|_2$.
2. Implementasikan operasi nomor 1 dengan list dan NumPy array. Catat perbedaan perilaku operator.
3. Untuk dua matriks 2×2, hitung manual $A \odot B$ dan AB, lalu bandingkan `A*B` dan `A@B`.
4. Tulis `matmul_manual(A,B)` dengan tiga loop dan pemeriksaan dimensi.
5. Uji perkalian matriks 2×3 dengan 3×2 dan bandingkan dengan NumPy.
6. Buat transpose matriks dengan loop bersarang.
7. Buat transpose dengan list comprehension dan dengan `np.array(A).T`.
8. Implementasikan invers matriks 2×2 dari rumus manual. Tolak matriks singular.
9. Bandingkan invers manual dengan `np.linalg.inv` dan verifikasi $AA^{-1} \approx I$.
10. Selesaikan SPL 2×2 dengan invers manual dan `np.linalg.solve`.
11. Hitung determinan sebelum menyelesaikan SPL. Jelaskan arti determinan nol terhadap keunikan solusi pada konteks ini.
12. Untuk solusi $\hat{x}$, hitung residual $r = b - A\hat{x}$ dan $\|r\|_2$.
13. Ganggu satu komponen solusi sebesar 10^{-3} dan bandingkan norma residual.
14. Selesaikan sistem 3×3 dari LKM dan verifikasi hasil melalui residual.
15. Bandingkan secara konseptual `np.linalg.solve(A,b)` dengan `np.linalg.inv(A) @ b`.
16. Untuk $A = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}$, hitung nilai eigen manual dari $\det(A - \lambda I) = 0$.
17. Gunakan `np.linalg.eig` dan verifikasi $Av \approx \lambda v$ dengan norma selisih.
18. Untuk $X = \begin{pmatrix} 1 & 2 \\ 2 & 1 \\ 4 & 3 \end{pmatrix}$, hitung rata-rata kolom, centered matrix, dan $X_c^T X_c$.
19. Kerjakan nomor 18 sekali dengan loop/list dan sekali dengan NumPy.
20. Buat data empat observasi dengan tiga fitur. Hitung Gram matrix $G = X_c^T X_c$ dan periksa bahwa G simetris.
21. Tulis fungsi yang menerima A, b dan mengembalikan solusi serta norma residual.
22. Buat tiga test case untuk fungsi pada nomor 21: identitas, sistem nontrivial, dan matriks singular.

Bab 9

Latihan Terpadu Menjelang UTS

Bagian ini digunakan setelah Praktikum 1-7. Soal sengaja mencampur prediksi, tracing, debugging, penulisan program, analisis algoritma, dan komputasi matriks.

Latihan terpadu

1. Prediksi nilai dan tipe beberapa ekspresi yang mencampur `int`, `float`, `bool`, dan `str`. Sertakan satu contoh yang menghasilkan error.
2. Tulis program yang menerima koefisien persamaan kuadrat dan mengklasifikasikan jenis akar.
3. Trace loop yang menggunakan `continue` untuk `None` dan `break` untuk nilai ekstrem.
4. Tulis nested loop yang menghasilkan pola segitiga dan hitung jumlah total iterasi dalam bentuk rumus.
5. Buat fungsi yang mengembalikan empat ringkasan statistik dan lakukan unpacking.
6. Diagnosa fungsi yang hanya melakukan `print` tetapi hasilnya dipakai dalam ekspresi aritmetika.
7. Trace scope lokal dan global pada fungsi dengan nama variabel yang sama.
8. Gunakan fungsi sebagai argumen untuk menerapkan tiga transformasi pada satu nilai.
9. Tulis fungsi rekursif dari suatu relasi rekurensi dan tunjukkan base case.
10. Tulis pseudocode Euclid dan trace untuk satu pasangan bilangan.
11. Bentuk matriks sederhana dengan nested loop dan konversi ke NumPy.
12. Gunakan slicing untuk menghitung beda hingga dari dua array.
13. Temukan bug pada fungsi rata-rata yang memiliki `return` di lokasi salah.
14. Tulis `assert` untuk fungsi jarak yang mencakup kondisi batas.
15. Tentukan apakah tiga loop berbeda berhenti untuk semua input positif dan berikan alasan.
16. Turunkan iterasi Newton untuk $\sqrt{n}$ dari Newton-Raphson umum, lalu tulis implementasinya.
17. Tentukan Big-O dua loop bersarang, dua loop berurutan, dan loop yang membagi ukuran dengan dua.
18. Untuk dua model biaya polinomial, tentukan orde dan cari titik silang numerik.
19. Trace tiga pass selection sort pada list yang diberikan.
20. Trace binary search untuk key yang ditemukan dan key yang tidak ditemukan.
21. Gambar dua atau tiga level awal merge sort untuk list panjang 16.
22. Jelaskan perbedaan analisis kompleksitas dan benchmark empiris.
23. Hitung operasi vektor, norma, dan dot product secara manual lalu verifikasi dengan NumPy.
24. Bandingkan `A*B` dan `A@B` pada dua matriks 2×2.
25. Selesaikan SPL dan hitung norma residual.
26. Hitung nilai eigen matriks simetris 2×2 dan verifikasi pasangan eigen numerik.
27. Centering sebuah matriks data dan hitung $X_c^T X_c$; jelaskan mengapa hasilnya simetris.

Bab 10

Praktikum 8: Regresi Linear Berbasis Aljabar Linier

Tujuan praktikum

Mahasiswa mampu membentuk matriks desain, menghitung parameter least squares sederhana, membuat prediksi, menghitung residual, dan memvisualisasikan hasil regresi.

Untuk model $y_i = \beta_0 + \beta_1 x_i + \varepsilon_i$, bentuk matriks

$$y = X\beta + \varepsilon,$$

dan cari β melalui persamaan normal $(X^T X)\beta = X^T y$ untuk tujuan pembelajaran aljabar linier.

```
import numpy as np

x = np.array([1., 2., 3., 4., 5.])
y = np.array([2.1, 3.9, 6.2, 8.0, 10.1])
X = np.column_stack([np.ones(len(x)), x])
beta = np.linalg.solve(X.T @ X, X.T @ y)
yhat = X @ beta
residual = y - yhat
print(beta)
print(np.linalg.norm(residual))
```

Latihan Praktikum 8

1. Bentuk matriks desain untuk regresi dengan intercept.
2. Hitung $X^T X$ dan $X^T y$ secara manual untuk dataset kecil.
3. Selesaikan parameter dengan `np.linalg.solve`.
4. Hitung prediksi dan residual.
5. Hitung SSE, RMSE, dan R^2.
6. Buat scatter plot data dan garis regresi.
7. Bandingkan hasil dengan `numpy.polyfit`.
8. Ubah skala x dan amati perubahan parameter.
9. Tambahkan satu outlier dan amati perubahan model.
10. Jelaskan hubungan residual dengan konsep jarak dari data ke model.

Bab 11

Praktikum 9: Klasifikasi k-Nearest Neighbor

Tujuan praktikum

Mahasiswa mampu menggunakan jarak Euclid untuk mencari tetangga terdekat, melakukan voting kelas sederhana, dan mengevaluasi prediksi pada dataset kecil.

```python
import numpy as np

def jarak(x, y):
    return np.linalg.norm(x - y)
```

Latihan Praktikum 9

1. Hitung jarak sebuah titik uji ke semua data latih.
2. Urutkan indeks berdasarkan jarak.
3. Implementasikan k-NN untuk $k = 1$ dan $k = 3$.
4. Uji pengaruh perubahan skala salah satu fitur.
5. Standardisasi fitur lalu ulangi klasifikasi.
6. Buat confusion matrix sederhana secara manual.
7. Hitung akurasi pada data uji kecil.
8. Uji beberapa nilai k dan bandingkan hasilnya.
9. Jelaskan apa yang terjadi jika terdapat voting seri.
10. Bandingkan implementasi manual dengan pustaka yang tersedia jika diizinkan.

Bab 12

Praktikum 10: k-Means dan Mini Proyek Data

Tujuan praktikum

Mahasiswa mampu memahami proses iteratif k-means, menentukan stopping rule sederhana, memeriksa perubahan centroid, dan menyusun mini proyek yang dapat dijalankan ulang.

Secara umum k-means mengulang dua langkah: menetapkan setiap titik ke centroid terdekat, lalu memperbarui centroid sebagai rata-rata anggota cluster. Iterasi berhenti ketika assignment atau centroid tidak lagi berubah secara berarti.

Latihan Praktikum 10

1. Lakukan satu iterasi k-means secara manual pada empat titik dan dua centroid awal.
2. Implementasikan assignment ke centroid terdekat.
3. Implementasikan pembaruan centroid.
4. Tambahkan stopping rule berdasarkan perubahan centroid.
5. Catat objective sum of squared distances pada setiap iterasi.
6. Uji beberapa inisialisasi centroid dan bandingkan hasilnya.
7. Visualisasikan cluster untuk data dua dimensi.
8. Jelaskan mengapa skala fitur memengaruhi k-means.
9. Susun folder mini proyek berisi kode, data, output, dan README.
10. Tulis rumusan masalah, input, output, algoritma, validasi, dan cara menjalankan ulang proyek.
11. Buat satu eksperimen tambahan yang mengubah parameter utama dan catat dampaknya.
12. Tulis ringkasan satu halaman yang menjelaskan apa yang dapat dan tidak dapat disimpulkan dari hasil komputasi.

Daftar Pustaka Singkat

1. Miller, B. dan Ranum, D. *Problem Solving with Algorithms and Data Structures Using Python*, Release 3.0.
2. Pine, D. J. *Introduction to Python for Science and Engineering.* CRC Press, 2019.
3. McKinney, W. *Python for Data Analysis: Data Wrangling with pandas, NumPy, and Jupyter*, 3rd ed. O'Reilly, 2022.
4. Dokumentasi Python: `https://docs.python.org/3/`
5. Dokumentasi NumPy: `https://numpy.org/doc/`